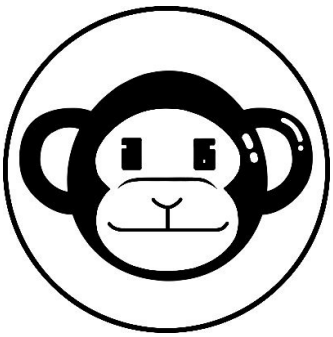


ape16z

Audit



Presented by:

OtterSec contact@osec.io

Ajay Kunapareddy d1r3wolf@osec.io



Contents

01 Executive Summary **2**

Overview 2

Key Findings 2

02 Scope **3**

03 Findings **4**

04 Vulnerabilities **5**

OS-A16-ADV-00 [low] | Deprecated methods in candy machine UI 6

OS-A16-ADV-01 [low] | Outdated dependencies in candy machine UI 7

05 General Findings **9**

OS-A16-SUG-00 | Length check for filenames in assets folder 10

OS-A16-SUG-01 | UploadDataArgs struct is unused 12

OS-A16-SUG-02 | Simplified logging 13

OS-A16-SUG-03 | Unnecessary error handling in sugar deploy 15

Appendices

A Program Files **16**

B Procedure **18**

C Implementation Security Checklist **19**

D Vulnerability Rating Scale **21**

© 2022 OtterSec LLC. All Rights Reserved.

01 | Executive Summary

Overview

ape16z engaged OtterSec to perform an assessment. This assessment was conducted between August 8th and August 26th, 2022.

Critical vulnerabilities were communicated to the team prior to the delivery of the report to speed up remediation. After delivering our audit report, we worked closely with the team over to streamline patches and confirm remediation.

We delivered final confirmation of the patches **[not yet delivered]**.

Key Findings

The following is a summary of the major findings in this audit.

- 6 findings total
- No vulnerabilities which could lead to loss of funds

© 2022 OtterSec LLC. All Rights Reserved.

02 | Scope

The source code was delivered to us in git repositories at github.com/metaplex-foundation/candy-machine-ui, github.com/metaplex-foundation/sugar, and github.com/DeepGreenAI/sugar-deployer. This audit was performed against commits `6c194a3`, `3a228a3`, and `7d2c47c` respectively.

There were a total of 5 targets included in this audit. A brief description of the programs is as follows. A full list of program files and hashes can be found in Appendix A.

Name	Description
candy-machine-ui	React app for minting NFTs from the deployed candy machine.
sugar/deploy	Sugar cli command for deploying the candy machine using deployed files.
sugar/upload	Sugar cli command for uploading NFTs to storage provided in config
sugar/withdraw	Sugar cli command for withdrawing the lamports from the candy machine.
sugar-deployer	Open source repo containing bash scripts for setting up the sugar CLI and deploying a candy machine on the Solana network.

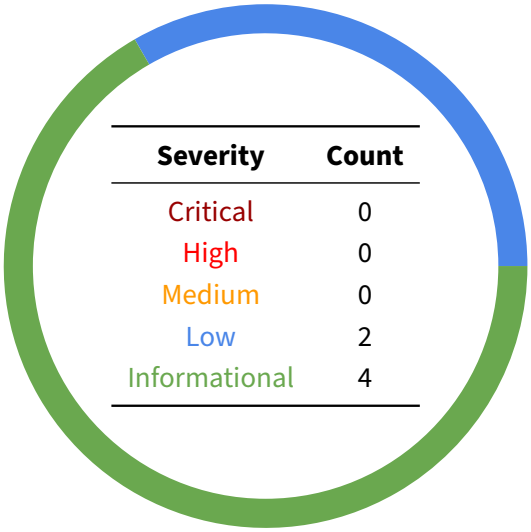
© 2022 OtterSec LLC. All Rights Reserved.

03 | Findings

Overall, we report 6 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings don't have an immediate impact but will help mitigate future vulnerabilities.

The below chart displays the findings by severity.



Severity	Count
Critical	0
High	0
Medium	0
Low	2
Informational	4

© 2022 OtterSec LLC. All Rights Reserved.

04 | Vulnerabilities

Here we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have **immediate** security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix D.

ID	Severity	Status	Description
OS-A16-ADV-00	Low	TODO	The web3.js connection methods used in the candy-machine-ui are deprecated.
OS-A16-ADV-01	Low	TODO	The candy machine UI uses many outdated package dependencies.

© 2022 OtterSec LLC. All Rights Reserved.

OS-A16-ADV-00 [low] | Deprecated methods in candy machine UI

Description

In the `candy-machine-ui` repository, `src/connection.tsx` uses deprecated connection methods from web3.js. The deprecated methods are:

1. `getParsedConfirmedTransaction`
2. `getRecentBlockhash`
3. `setSigners`

Remediation

Use new methods from web3.js instead of deprecated ones.

1. Use `getParsedTransaction` over `getParsedConfirmedTransaction`.
2. Use `getLatestBlockhash` over `getRecentBlockhash`.
3. Use `addSignature` over `setSigners`.

© 2022 OtterSec LLC. All Rights Reserved.

OS-A16-ADV-01 [low] | Outdated dependencies in candy machine UI

Description

The versions of packages used in `candy-machine-ui` are outdated, which might lead to malicious exploits through known vulnerabilities in the outdated (and insecure) versions.

```
# npm outdated
Package                              Current    Wanted   Latest
@babel/runtime                         7.18.3    7.18.9   7.18.9
@project-serum/anchor                  0.17.0    0.17.0   0.25.0
@solana/spl-token                       0.1.8     0.1.8    0.3.2
@solana/wallet-adapter-base             0.7.1     0.7.1   0.9.15
@solana/wallet-adapter-material-ui     0.13.1    0.13.1  0.16.13
@solana/wallet-adapter-react           0.13.1    0.13.1  0.15.17
@solana/wallet-adapter-react-ui         0.6.1     0.6.1   0.9.15
@solana/wallet-adapter-wallets         0.11.3    0.11.3   0.18.4
@solana/web3.js                        1.39.0    1.53.0   1.53.0
@testing-library/dom                   7.31.2    7.31.2   8.17.1
@testing-library/jest-dom              5.16.4    5.16.5   5.16.5
@testing-library/react                 11.2.7    11.2.7   13.3.0
@testing-library/user-event            12.8.3    12.8.3   14.4.3
@types/jest                           26.0.24   26.0.24   28.1.7
@types/node                          12.20.55  12.20.55  18.7.11
@types/react                          17.0.45   17.0.48  18.0.17
@types/react-dom                      17.0.17   17.0.17   18.0.6
@types/styled-components               5.1.25    5.1.26   5.1.26
@typescript-eslint/eslint-plugin       5.30.6    5.34.0   5.34.0
@typescript-eslint/parser              5.30.6    5.34.0   5.34.0
eslint                                 7.32.0    7.32.0   8.22.0
react                                  17.0.2    17.0.2   18.2.0
react-dom                              17.0.2    17.0.2   18.2.0
react-is                              16.13.1   16.13.1   18.2.0
react-scripts                          4.0.3     4.0.3    5.0.1
styled-components                      5.3.3     5.3.5    5.3.5
typescript                             4.7.3     4.7.4    4.7.4
web-vitals                             1.1.2     1.1.2    2.1.4
```

As of writing this report, there are 32 known vulnerabilities (not necessarily exploitable).

© 2022 OtterSec LLC. All Rights Reserved.

```
# npm audit
32 vulnerabilities (1 low, 1 moderate, 22 high, 8 critical)
```

Remediation

Update packages dependencies used in `candy-machine-ui` to their latest versions.

© 2022 OtterSec LLC. All Rights Reserved.

05 | General Findings

Here we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they do represent antipatterns and could introduce a vulnerability in the future.

ID	Status	Description
OS-A16-SUG-00	TODO	The length of the array of files matched by the regular expression used to find image/animation files should be validated.
OS-A16-SUG-01	TODO	The UploadDataArgs struct is not used anywhere in the sugar crate.
OS-A16-SUG-02	TODO	Logging can be simplified in a number of places.
OS-A16-SUG-03	TODO	In deployment, the treasury wallet will always exist, either specified or computed as an associated token address.

© 2022 OtterSec LLC. All Rights Reserved.

OS-A16-SUG-00 | Length check for filenames in assets folder

Description

In `sugar/src/upload/assets.rs`, the image/animation file for an NFT (index taken from the metadata filename) is found by filtering filenames in the assets folder via regular expression. For example, when the index is 0 we have

- image regex `"^0\.((jpg)|(gif)|(png))$"`
- animation regex `"^0\.((mp4)|(mov)|(webm))$"`

Although the resulting filename lists are validated to be non-empty, this allows for the scenario where there is more than one filename, e.g. `0.png` and `0.jpg`. This might lead to unexpected behavior.

sugar/src/upload/assets.rs RUST

```rust
let img_filenames = paths_ref
    .clone()
    .into_iter()
    .filter(|p| img_regex.is_match(p))
    .collect::<Vec<String>>();

let img_filename = if img_filenames.is_empty() {
```

sugar/src/upload/assets.rs RUST

```rust
let animation_filenames = paths_ref
    .clone()
    .into_iter()
    .filter(|p| animation_regex.is_match(p))
    .collect::<Vec<String>>();

...
let animation_filename = if !animation_filenames.is_empty() {
```

Remediation

Verify that the filename arrays contain exactly one element.

© 2022 OtterSec LLC. All Rights Reserved.

sugar/src/upload/assets.rs RUST

```rust
let img_filenames = paths_ref
    .clone()
    .into_iter()
    .filter(|p| img_regex.is_match(p))
    .collect::<Vec<String>>();

let img_filename = if img_filenames.len() != 1 {
```

sugar/src/upload/assets.rs RUST

```rust
let animation_filenames = paths_ref
    .clone()
    .into_iter()
    .filter(|p| animation_regex.is_match(p))
    .collect::<Vec<String>>();

...
let animation_filename = if !animation_filenames.len() != 1 {
```

© 2022 OtterSec LLC. All Rights Reserved.

OS-A16-SUG-01 | UploadDataArgs struct is unused

Description

In `sugar/src/upload/assets.rs`, the `UploadDataArgs` struct is defined but not used anywhere in the crate.

sugar/src/upload/assets.rs RUST

```rust
pub struct UploadDataArgs<'a> {
    pub bundlr_client: Arc<Bundlr<SolanaSigner>>,
    pub assets_dir: &'a Path,
    pub extension_glob: &'a str,
    pub tags: Vec<Tag>,
    pub data_type: DataType,
}
```

Remediation

Remove the `UploadDataArgs` struct definition.

© 2022 OtterSec LLC. All Rights Reserved.

OS-A16-SUG-02 | Simplified logging

Description

In `sugar/src/upload/process.rs`, the program logs progress in terms of steps towards completion. It repeatedly computes the total number of steps during each logging statement, which is redundant. An example is given below.

sugar/src/upload/process.rs RUST

```
println!(
    "\n{} {}Initializing upload",
    style(if indices.animation.is_empty() {
        "[2/4]"
    } else {
        "[2/5]"
    })
    .bold()
    .dim(),
    COMPUTER_EMOJI
);
```

Remediation

Compute the total number of steps initially, then reuse this quantity.

sugar/src/upload/process.rs RUST

```
let total_steps = if indices.animation.is_empty() { 4 } else { 5 };
println!(
    "\n{} {}Initializing upload",
    style(format!("[2/{}]", total_steps))
    .bold()
    .dim(),
    COMPUTER_EMOJI
);
```

Description

In `sugar/src/upload/process.rs`, the `if` condition contained in `println!` is unnecessary. Notice that the `indices.animation` array is ensured to be non-empty in the outer if condition.

© 2022 OtterSec LLC. All Rights Reserved.

sugar/src/upload/process.rs RUST

```rust
if !indices.animation.is_empty() {
    println!(
        "\n{} {}Uploading animation files {}",
        style("[4/5]").bold().dim(),
        UPLOAD_EMOJI,
        if indices.animation.is_empty() {
            "(skipping)"
        } else {
            ""
        }
    );
}
```

Remediation

Remove the inner if condition.

sugar/src/upload/process.rs RUST

```rust
if !indices.animation.is_empty() {
    println!(
        "\n{} {}Uploading animation files {}",
        style("[4/5]").bold().dim(),
        UPLOAD_EMOJI,
        ""
    );
}
```

© 2022 OtterSec LLC. All Rights Reserved.

OS-A16-SUG-03 | Unnecessary error handling in sugar deploy

Description

Shown below is a snippet of code which parses `spl_token_account` for the candy machine's treasury wallet. Notice that the highlighted else case is redundant, as `spl_token_account_figured` value cannot be None.

sugar/src/deploy/process.rs RUST

```
let spl_token_account_figured = if
    ↪  config_data.spl_token_account.is_some() {
    config_data.spl_token_account
} else {
    Some(get_associated_token_address(&program.payer(), &spl_token))
};
...
if let Some(token_account) = spl_token_account_figured {
    // validates the spl token wallet to receive proceedings from SPL
    ↪  token payments
    check_spl_token_account(&program, &token_account.to_string())?;
    token_account
} else {
    return Err(anyhow!(
        "If spl-token is set, spl-token-account must also be set"
    ));
}
```

Remediation

Refactor the above code to eliminate the else case for `spl_token_account_figured`.

sugar/src/deploy/process.rs RUST

```
let token_account = config_data.spl_token_account
    .unwrap_or(get_associated_token_address(&program.payer(),
    ↪  &spl_token));
...
check_spl_token_account(&program, &token_account.to_string())?;
token_account
```

© 2022 OtterSec LLC. All Rights Reserved.

A | Program Files

Below are the files in scope for this audit and their corresponding SHA256 hashes.

candy-machine-ui

```
package.json                                72dd15bcbc3f5733fdabc56e110341da73a72453cf1ad9eab2ad0562980a2e0c
tsconfig.json                               cb9cee8200f8070c9e36ee74299938966d5a10718171bb671a95829b8a6b06a5
yarn.lock                                   2810c5351c4ae72a817017a24614448e6330f858b58fdec980aedd397ca0007f
src
  App.css                                   37ffacc0c207c8639c23fee39ba452244f77279e1a0f07aa5feb02fe028b9be8
  App.tsx                                   2a1309efc251489bb7f7f969af82adfda6b05685a63d9ea50834de9d3745f693
  Home.tsx                                  bc1b975295a87201da7e47c6f0739b77e3ee66f2d8ad46ceecefc77ce0ed2275
  MintButton.tsx                            ba6b076989e7738289e8b233bd371b267db3bf9432da37af6292fcc8c154741b
  MintCountdown.tsx                         d96a8cd64f96ef3ea7d10831bf11382689521e9b2be2ee5a89e3c9a1dd126266
  candy-machine.ts                          10e1f8adf3c38649026a8d4f348f4825fd114c2563115f7ac2144a4c39911ea2
  connection.tsx                            fec7045d6f1b3d1eddc99a2fef2fa5f9ee7a15a4bbad1804af32389ccee1068f
  index.css                                 01298623d859c7314670a631346c19d97170803fdcee595f3ef27c508f575ef4
  index.tsx                                 8c6467111eff7e08fc178428271f632ba567d7d3f39df8a07310fb7ea8f8d7d0
  logo.svg                                  6000b0e9b0b05b3f112de04f0d039768a1db63588ff9b6ef7099dbd71632f383
  react-app-env.d.ts                        57eda4c4c04a1dca45c62857326882ce9cc948c4b52973c0e3c3b7e4c3fa3990
  reportWebVitals.ts                        780f64898ef44a68c0e45687a54528aa19e2c56d9c83940dee7a74c4a3430bfd
  setupTests.ts                             c261337d5c5512bf5d4d890646d4e10e5263de8d2c78cc48036eb968d1447cc0
  utils.ts                                  d6f6db1e70c3e64fcb0cfd3213ee193ba186b416d1eb6651e016111e7d8fec68
```

sugar/deploy

```
collection.rs                               aee9823af3f85cf57a0f7a42113b8a5fac12dee5f9a6292a8a72205a31cf2d9e
config_lines.rs                             4e3686f2b380093b38664fb926fa05260ef38087e5546ebaf6e31c984e24552d
errors.rs                                   fdd07133ac131e4063aa7804c9effb8e7d0730739671518027759 83c5bb4e6cd
initialize.rs                               b4b259620d37230c6ca5df050d2f9d8bb3ffe5a1480f57a52678265ffd4d1555
mod.rs                                      a6dfd0f8cf8ba6c8d0701212845e0b578486fcb0eea4e244c2b6184ba64b97ad
process.rs                                  18644f8182c93c4f9dbe7187bdbd878c33db7b540833c4c38e9f4fc357e5ad48
```

sugar/withdraw

```
mod.rs                                      11d96d7134bd4154fb72b484c787d2e399369cf83e78bf05b6c4bb311fc989f8
process.rs                                  5c75cf89545f2e26f606a53280208c2d9b253b2bea16d8d52a0780d09ae84cce
```

sugar/upload

```
assets.rs                                   ef1f9fb1e06deec294c7c710699ca111bf3256c4e6e045f05ca4e41f4fa8ebaf
errors.rs                                   8ecfd9d0d7ac4422124c5cbe21c248dbbe7fd6f424158b1455894f7bfcda47c8
mod.rs                                      f589f57aad7e5d77f6b47072c768ab6318bd575ffd0479dbc81dfbba4179b7b0
process.rs                                  41e6ecf146be666ae1b78f48303a13cd93e611181dcfd312e0fb3c8374edc1c7
uploader.rs                                 55e226df82c644f2ce8674a2b9f948102ef91b91ff5cc5699a2aebc8ea04195b
methods
  aws.rs                                    256787bf65928ba9dfbf9a54422cc26e33e662bbaccd6b42a03d6316b62f076e
  bundlr.rs                                 7d023ea5966ccc7ced5a545a92a6575ec30276bfa93cff0f652ceaa7490abe10
  mod.rs                                    074d83079c0f448c31182ce8002ec814f29db048d843503858958b50b95a859f
  nft_storage.rs                            16bd6ec3ab381e0e156a44492bfcbc92317ebe4a06c76396280052ff66644feb
  shdw.rs                                   45fb6f208698e439a35b55f8a86bbd2647676d42187bb0d9ab997c4d57902b5a
```

© 2022 OtterSec LLC. All Rights Reserved.

sugar-deployer

1-install-sugar-solana.bash	92666ad3c12f4c7de5f57a944d54adf4115fa2ce69a66f6ea1331b2bd2295871
2-setup-devnet.bash	545355605b114fc6340d653dcc09249ad900a8500e4a17d73926021b46ddb899
2-setup-mainnet.bash	ea716263bed337e9bd28598c9a73d714face8545faa462768745a0ddde6c64bd
3-deploy-candy-machine.bash	01ffd260065a926e7b85b23b2ff942e4fa58fb614bbe53dc2dca8b5a2a050ff0
4-update-candy-machine.bash	10d46b2e206d9edfc7002606873155d2a9656fce60deb913837a9feabdb16cb6
5-withdraw-rent-and-destroy-candy-machine.bash	2bd98012851c665d2097f6be5106535ab2349f2f0ea60b42818edecf642e8656
config.json	217c7bf4518216d7b5df7dd3b192f217dedf68e8897a0378bfc7f7338971274d

© 2022 OtterSec LLC. All Rights Reserved.

B | Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an onchain program. In other words, there is no way to steal tokens or deny service, ignoring any Solana specific quirks such as account ownership issues. An example of a design vulnerability would be an onchain oracle which could be manipulated by flash loans or large deposits.

On the other hand, auditing the implementation of the program requires a deep understanding of Solana's execution model. Some common implementation vulnerabilities include account ownership issues, arithmetic overflows, and rounding bugs. For a non-exhaustive list of security issues we check for, see Appendix C.

Implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to get a comprehensive understanding of the program first. In our audits, we always approach any target in a team of two. This allows us to share thoughts and collaborate, picking up on details that the other missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2022 OtterSec LLC. All Rights Reserved.

C | Implementation Security Checklist

Unsafe arithmetic

Integer underflows or overflows	Unconstrained input sizes could lead to integer over or underflows, causing potentially unexpected behavior. Ensure that for unchecked arithmetic, all integers are properly bounded.
Rounding	Rounding should always be done against the user to avoid potentially exploitable off-by-one vulnerabilities.
Conversions	Rust as conversions can cause truncation if the source value does not fit into the destination type. While this is not undefined behavior, such truncation could still lead to unexpected behavior by the program.

Account security

Account Ownership	Account ownership should be properly checked to avoid type confusion attacks. For Anchor, the safety of unchecked accounts should be clearly justified and immediately obvious.
Accounts	For non-Anchor programs, the type of the account should be explicitly validated to avoid type confusion attacks.
Signer Checks	Privileged operations should ensure that the operation is signed by the correct accounts.
PDA Seeds	PDA seeds are uniquely chosen to differentiate between different object classes, avoiding collision.

© 2022 OtterSec LLC. All Rights Reserved.

Input validation

Timestamps	Timestamp inputs should be properly validated against the current clock time. Timestamps which are meant to be in the future should be explicitly validated so.
Numbers	Sane limits should be put on numerical input data to mitigate the risk of unexpected over and underflows. Input data should be constrained to the smallest size type possible, and upcasted for unchecked arithmetic.
Strings	Strings should have sane size restrictions to prevent denial of service conditions
Internal State	If there is internal state, ensure that there is explicit validation on the input account's state before engaging in any state transitions. For example, only open accounts should be eligible for closing.

Miscellaneous

Libraries	Out of date libraries should not include any publicly disclosed vulnerabilities
Clippy	cargo clippy is an effective linter to detect potential anti-patterns.

© 2022 OtterSec LLC. All Rights Reserved.

D | Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings can be found in the General Findings section.

Critical	Vulnerabilities which immediately lead to loss of user funds with minimal preconditions Examples: • Misconfigured authority/token account validation • Rounding errors on token transfers
High	Vulnerabilities which could lead to loss of user funds but are potentially difficult to exploit. Examples: • Loss of funds requiring specific victim interactions • Exploitation involving high capital requirement with respect to payout
Medium	Vulnerabilities which could lead to denial of service scenarios or degraded usability. Examples: • Malicious input cause computation limit exhaustion • Forced exceptions preventing normal use
Low	Low probability vulnerabilities which could still be exploitable but require extenuating circumstances or undue risk. Examples: • Oracle manipulation with large capital requirements and multiple transactions
Informational	Best practices to mitigate future security risks. These are classified as general findings. Examples: • Explicit assertion of critical internal invariants • Improved input validation • Uncaught Rust errors (vector out of bounds indexing)

© 2022 OtterSec LLC. All Rights Reserved.